HTML, CSS and Bootstrap

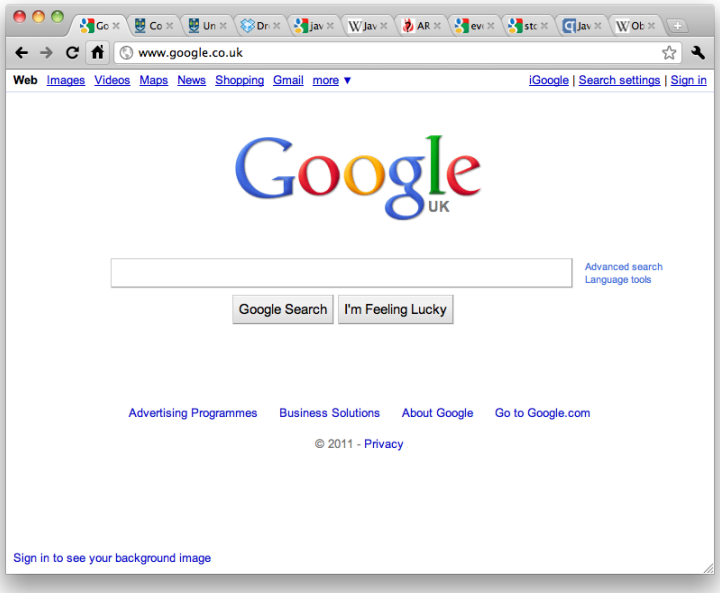
Internet Technology

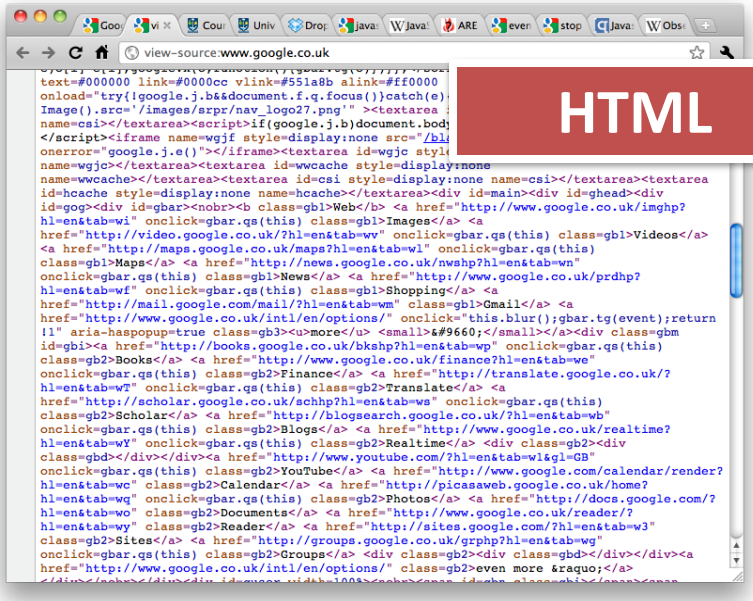
ITECH

By the end of Week 3...

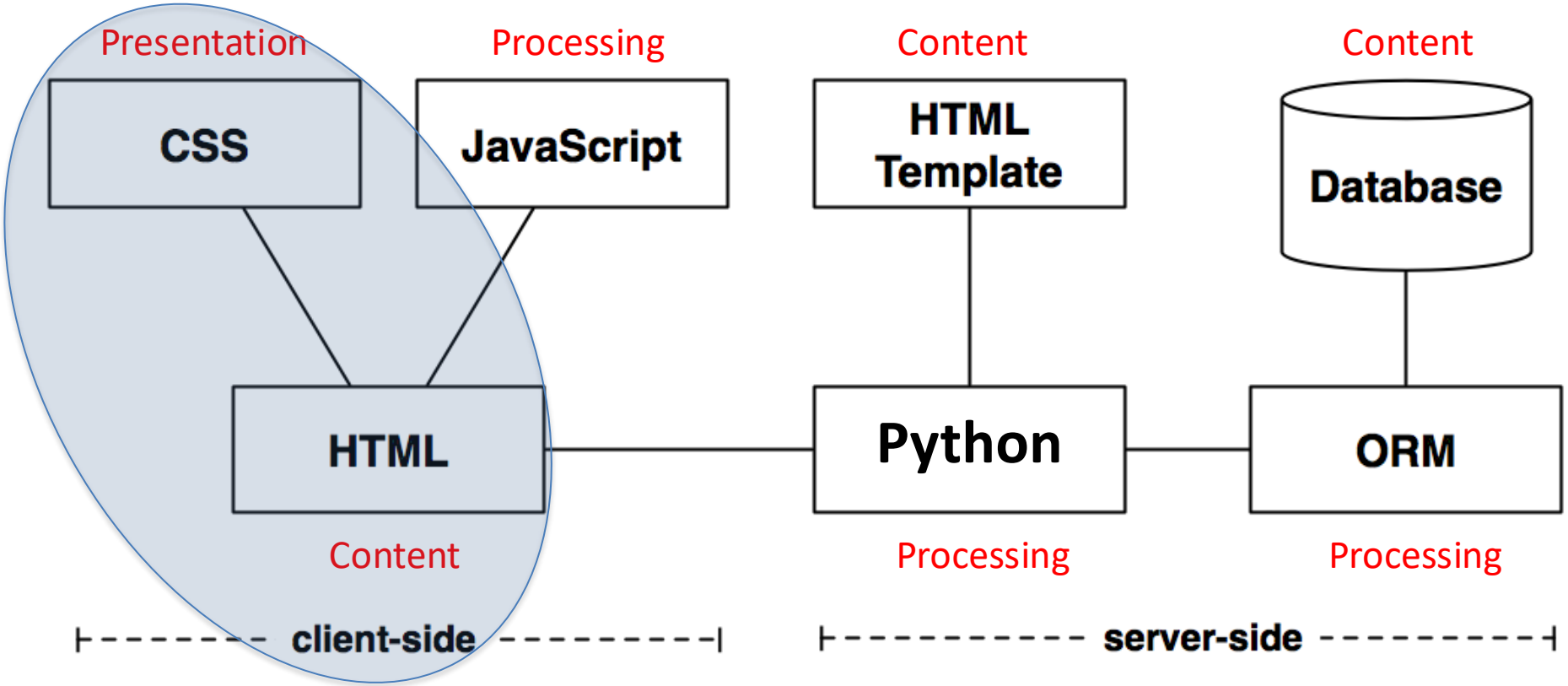
- By the end of Week 3, you should be able to:
  1. Construct Structured HTML Documents
  2. Apply CSS for Styling and Layout
  3. Understand how front-end frameworks (e.g. Bootstrap) can ease web development
  4. Understand Django Database modeling, and using data in templates (own study)
  5. Learn about accessibility and sustainability in web design and development

What is in a page?

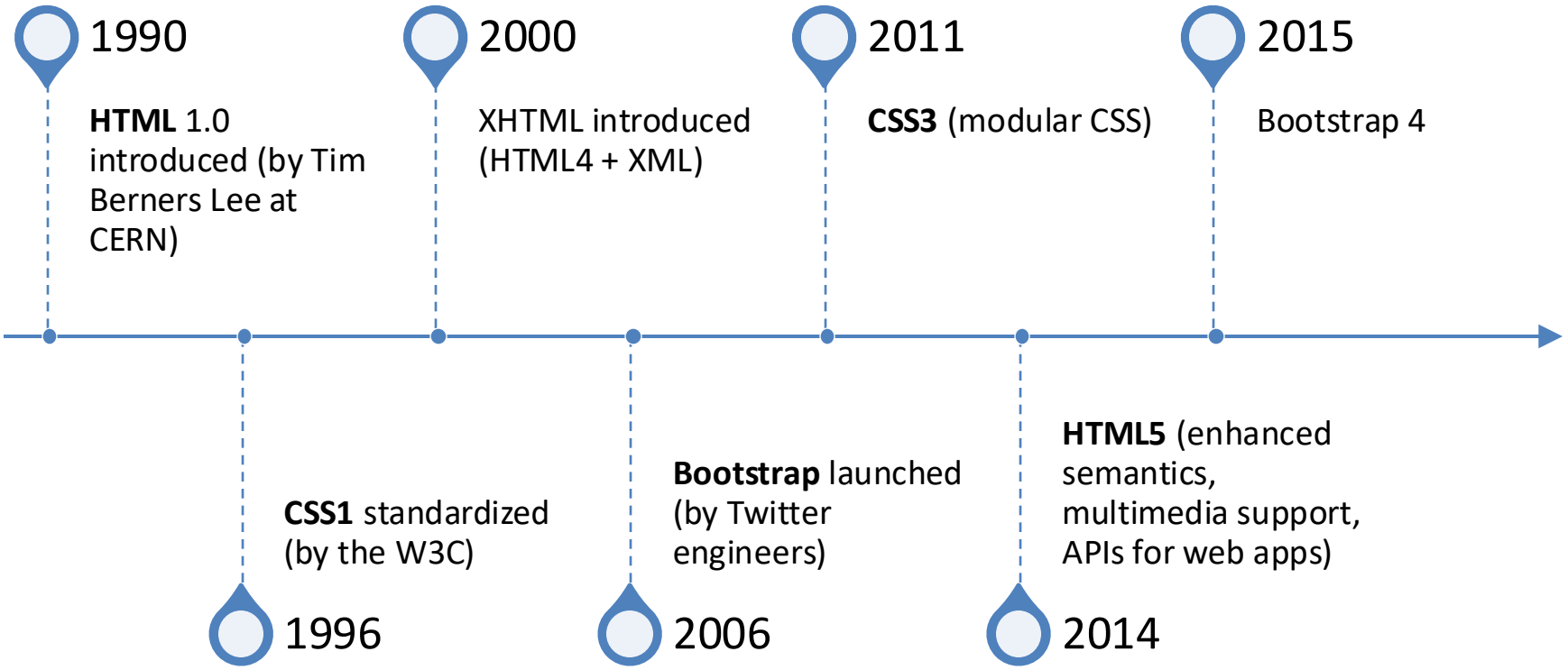




Client-side


Presentation
Processing
Content
Content
CSS
JavaScript
HTML Template
Database
HTML
Python
ORM
Content
Processing
Processing
client-side
server-side

HTML, CSS, Bootstrap: Evolution



Books

- (legacy) Beginning HTML and CSS / Rob Larsen
- The Absolute Beginner's Guide to HTML and CSS / Kevin Wilson
- The HTML and CSS Workshop : Learn to Build Your Own Websites and Kickstart Your Career As a Web Designer or Developer / Lewis Coulson, Brett Jephson, Rob Larsen, Matt Park, and Marian Zburlea

HTML

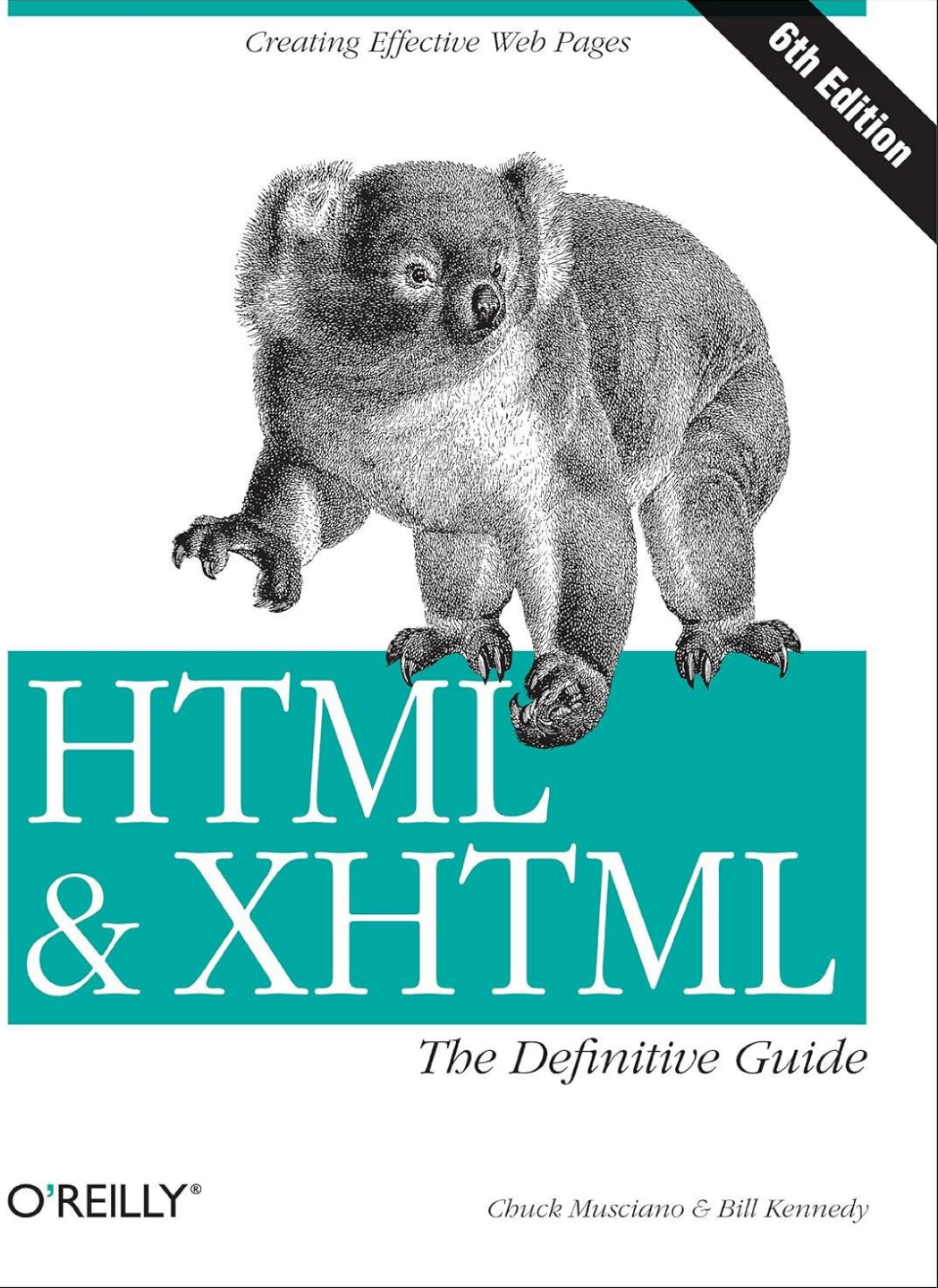
What is HTML?

- HTML stands for **HyperText Markup Language**
- It is the language web browsers use to interpret what gets displayed when you view a website
  - **Hypertext** is the link between documents
  - A **mark-up language** is a set of tags which describe document content
- HTML documents (webpages) contain HTML mark-up tags and plain text

(X)HTML Guide & Reference

or visit

https://www.w3schools.com/html/



Basic HTML Example: Tags

```
<html>
<head>
 <title> The title</title>
 </head>
 <body>

 <h1>ISD</h1>

 <p>My first paragraph.</p>

 </body>
 </html>
```

← Begin HTML now

← End HTML now

Tags:

- Keywords (tag names) surrounded by <>
- Normally have opening and closing tags

Plain text:

- Between tags
- Is the content displayed in the browser

Basic HTML Example: HTML Document Structure

```
<html>
  <head>
    <title> The title</title>
  </head>
  <body>

    <h1>ISD</h1>

    <p>My first paragraph.</p>

  </body>
</html>
```

HTML Document Structure:

- Tags are **nested**
- Starts with **<html>** tag
- **<head>** tag contains information about the document such as title and other things
- **<body>** tag contains the html to be displayed

Basic HTML Example: HTML Elements

```
<!DOCTYPE html>
<html>
    <head>
      <title>The title</title>
    </head>
    <body>

      <h1>Things to do</h1>

      <p>My first paragraph.</p>

    </body>
</html>
```

Elements

- Everything from an opening tag to a closing tag is called an **element**
- The plain text between opening and closing tags is called the **element content**

Basic HTML Example: Empty Elements

```
<!DOCTYPE html>
<html>
    <head>
      <title>The title</title>
    </head>
    <body>

      <h1>Things to Do</h1>
       Make a HTML page<br>
       Add a paragraph
      <p>My first paragraph.</p>

    </body>
</html>
```

Empty Elements

- There are some tags which have no content
- They also have no end tag
  - e.g.
 which forces a line break

Basic HTML Example: Headers

```
<!DOCTYPE html>
<html>
  <head>
    <title> The title</title>
  </head>
  <body>

    <h1>Things to To</h1>

    <p>My first paragraph.</p>

  </body>
</html>
```

Headers:

- **<h1>** - “header 1”
  - Used just once
  - Defines the most important heading
  - Search engines use <h1> to defer the content of your web pages
  - There are h1,…,h6 headers. <h1> is the most important.

Basic HTML Example: Paragraph

```
<!DOCTYPE html>
<html>
    <head>
      <title>The title</title>
    </head>
    <body>

      <h1>ISD</h1>

      <p>My first paragraph.</p>

    </body>
</html>
```

Paragraph:

- **<p>** is the paragraph tag
- Browsers add space (margin) before and after each <p> element
- They ignore your own formatting

Other useful HTML tags

- List items / unordered list

```
<ul>
    <li> List item one</li>
    <li> List item two</li>
</ul>
```

- Div elements let you create sections to divide up the page in different ways when coupled with CSS.

```
<div> </div>
```

HTML First

HTML First: A writing style for web development -- not necessary the best

- If you can do it with HTML, use HTML
- If you can’t do it with HTML, use CSS
- If you can’t do it with HTML or CSS, use Javascript

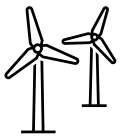
HTML AND WEB SUSTAINABILITY

Web Sustainability

Carbon Footprint?

- Total greenhouse gases emitted (tons of CO_2)
- Energy sources:

Brown energy: High emissions



Green energy: Low emissions

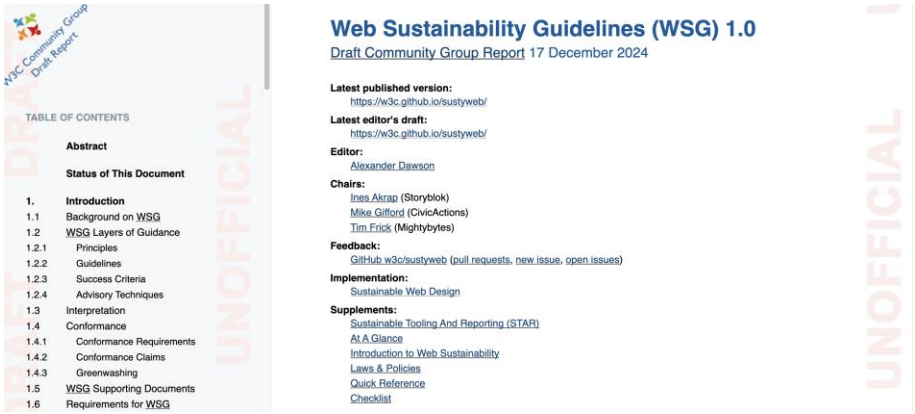
Increased data = Higher emissions

- **Data Centers:** High energy use to store data
- **Networks:** High energy use to transfer data
- **User devices:** High emergy use to load/interact with data

The internet produces ~4% of the world's emissions (and increasing)

Web Sustainability Guidelines

- Web Sustainability Guidelines: a working draft from the W3C



W3C Community Group Draft Report

TABLE OF CONTENTS

UNOFFICIAL

Web Sustainability Guidelines (WSG) 1.0

Draft Community Group Report 17 December 2024

Latest published version:
https://w3c.github.io/sustyweb/

Latest editor's draft:
https://w3c.github.io/sustyweb/

Editor:
Alexander Dawson

Chairs:
Ines Akrap (Storyblok)
Mike Gifford (CivicActions)
Tim Frick (Mightybytes)

Feedback:
GitHub w3c/sustyweb (pull requests, new issue, open issues)

Implementation:
Sustainable Web Design

Supplements:
Sustainable Tooling And Reporting (STAR)
At A Glance
Introduction to Web Sustainability
Laws & Policies
Quick Reference
Checklist

UNOFFICIAL

24 guidelines relevant to Web Development (Section 3): https://sustainablewebdesign.org/guideline-categories/web-development/

” If the Internet was a country, it would be the 4th largest polluter” – Sustainable Web Manifesto

Products and services should:

- Use clean energy
- Use the least amount of energy
- Be accessible
- Not exploit/mislead users
- Support an economy for people and the planet
- Function in times and places where people need them the most

Web Sustainability and HTML

Guideline #3.2: **Minify** your HTML, CSS, and Javascript

- No value in whitespace
- Improves page loading times

Web Sustainability and HTML

Guideline #3.8: Use **HTML elements** correctly

- Bloated markup wastes data, increases energy
- Deprecated code is not optimized

Deprecated HTML Elements

Deprecated Element	Instead, use:
basefont, font	CSS font family, font size, color
<dir>	,
<strike>	CSS text-decoration
	CSS text-align:center
<frame>	<iframe>, <frameset>

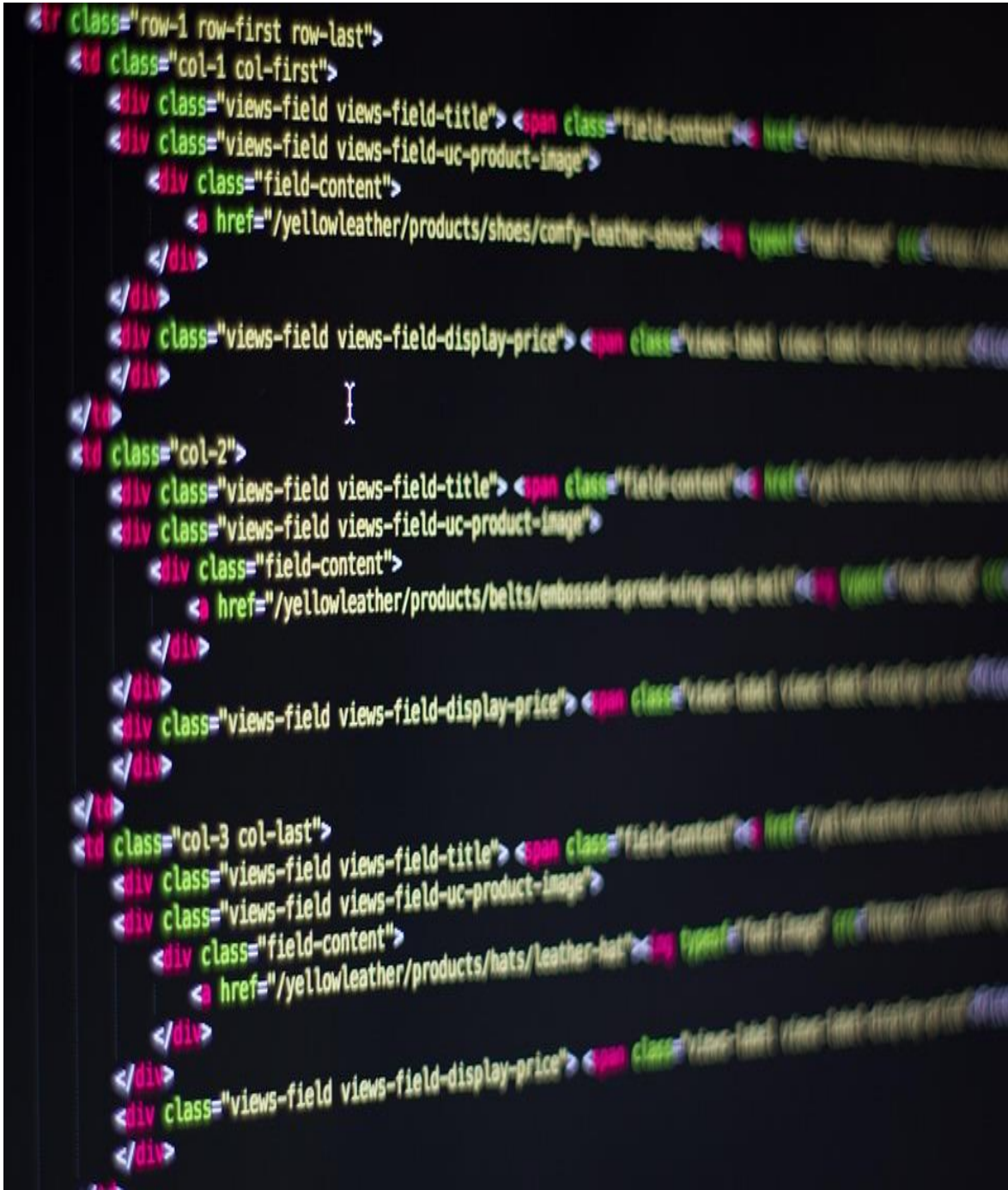
Useful sources:

- ObsoHTML: A tool to find Obsolete HTML
- https://blog.logrocket.com/deprecated-html-elements-and-what-to-use-instead
- https://css-tricks.com/why-do-some-html-elements-become-deprecated/

Deprecated HTML

Only 0.5% of websites use valid HTML!

https://meiert.com/en/blog/html-conformance-2024/



CASCADING STYLE SHEETS

Style on the Web

- **Decisions of Style:** Most aspects about any element of a web page can be controlled:
  - position, color, size, font etc
- This can be achieved in any number of ways:
  - by describing the page in XML and then using XSL to generate formatted XHTML
  - by using Cascading Style Sheets in combination with XML
  - by using **Cascading Style Sheets** in combination with **XHTML**

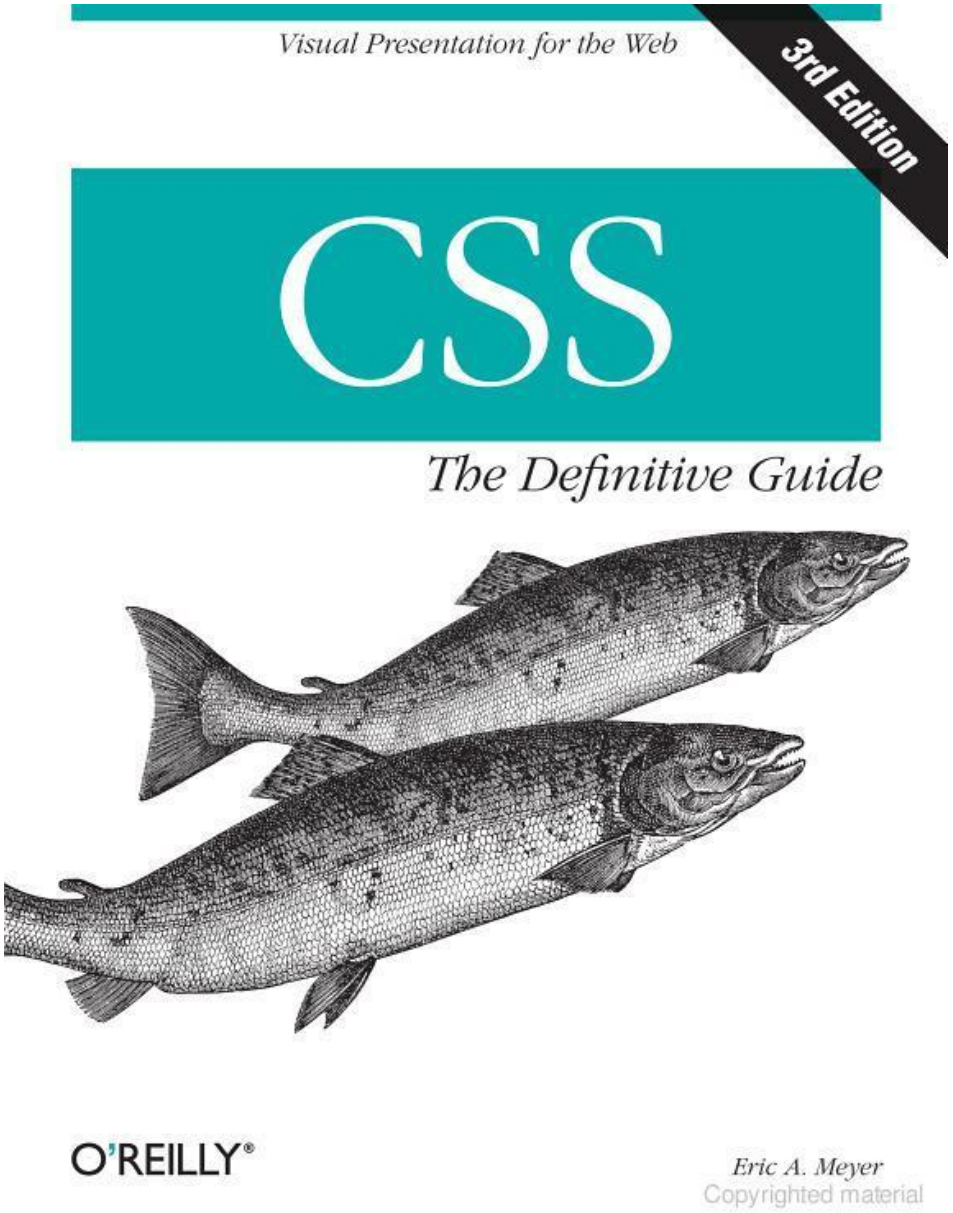
Benefits of CSS

- Cascading Style Sheets (CSS) is a method of **separating a document's structure and content from its presentation** – remember separation of concerns?
- CSS allows for a much **richer document appearance** than (X)HTML alone
- CSS can save time as the appearance of the entire document can be **created and changed in just one place**
- CSS can improve load times as it **compactly stores the presentation concerns** of a document in one place instead of being repeated throughout the document

CSS Guide & Reference

or visit

https://www.w3schools.com/css/



Cascading Style Sheets

- Stylesheets describe the rendering of html elements
  - they specify some style for **individual elements** or **all elements** of a particular kind
  - A CSS consists of a set of **formatting rules,** which are specified in the following way:

```
selector {
    property1: value1;
    property2: value2;
    …
}
```

```
h3 {
    color: yellow;
    size: 18px;
    …
}
```

  - **selector** indicates the element (or set of)
  - **property** refers to the stylistic aspect
  - **value** is the specific configuration.

CSS: How to Find and Apply Pattern

```
p {
    font-size: 12pt;
    font-face: “Verdana”; }

h1, h2, h3 {
    color: red;
    font-size: 18px; }

*{ text-align: left; }

#menu a {
    padding: 45px 25px 0px 25px;
    border: none;
    height: 80px; }
```

Apply to all <p> elements

Apply to all <h1>, <h2>, <h3> elements

Apply to all elements

Apply to all <a> elements with id=“menu”

CSS Values and Units

- **Units** affect the colors, distances, and sizes of a whole host of properties of an element's style
  - **Numbers**
    - can be integers or real numbers
  - **Percentages**
    - real number followed by %
    - generally relative to some other number
      - e.g. font-size:90% of the default or inherited value
  - **Color**
    - named color (e.g. 'red'), functional rgb(0,0,255), or hexadecimal RGB codes (#FF0000)

CSS Length Units

- Absolute Length Units
  - inches (**in**), centimeters (**cm**), millimeters (**mm**), points (**pt** – 72pt to 1 inch), and picas (**pi** – 1pi = 12pt)
- Relative Length Units
  - **em** is relative to the given font-size value
    - e.g. font-size is 14px, 1em = 14px
  - **ex** is relative to the size of a lowercase x for the given font family
  - **px** is (should be) the size of a pixel on the monitor
    - px is generally the recommended unit to use

Using CSS with (X)HTML

Inline CSS Specification

- **Inline**: within HTML
  - We add CSS syntax to one particular element
  - We use the **style** attribute in an HTML tag

```
<h3 style="color: yellow; font-size: 18pt">
```

- **Inline CSS** only affects this element, and others of the same type are not affected.
  - Useful to **override** existing style
  - but**...breaks** the **separation** of **content and presentation**

Embedded CSS Specification

- **Embedded**: In the <head> section of an HTML document
  - We add CSS style rules
  - The CSS rules will be applied to the **entire document**

```
<html>
    <head>
        <style>
            h3 { color: yellow; font-size: 18pt;  }
        </style>
    </head>
    <body>
        … <h3> This will be in yellow font size 18 </h3> …
    </body>
</html>
```

External CSS Specification

- **External**: In a separate document
  - The file extension is ".css"
  - The specified style can be shared by several pages

```
<head>
    <link rel="stylesheet" href="master.css" type="text/css">
</head>
```

- This is generally **the best method** in terms of:
  - Separation of concerns
  - Maintenance
  - Performance*

Specialization of Presentation

- **Class** and **ID selectors** can be used for finer control
- This involves more planning/effort with document markup
  - But can result in a better user experience
  - It is also very important for manipulating elements in Javascript
  - The effort also pays off if you use libraries like JQuery
- **Class selectors:** work on a set of specified elements through the **class attribute**
- **ID selectors:** provide a way to stylize unique elements through the **id attribute**

Class Selectors

- Class selector allow you to style items with the same (X)HTML element differently
- They work when the **class attribute** of an HTML tag is given a **name**
- The **dot (.)** operator is used to define the class

```
<style>
      *.warning {font-weight: bold;}
</style>

<p class="warning">This text will be displayed in bold.</p>

<p>This text will NOT be displayed in bold.</p>
```

ID Selectors

- Similar to class but they define a special case for an element
  - IDs are meant to be unique and only used once
  - However, browsers are not particularly fussy about enforcing the uniqueness of identifiers
- The **hash symbol (#)** is used to specify a unique ID

```
<style>
      #first-para { font-weight: bold; }
</style>

<p id="first-para">This paragraph will be bold-faced</p>

<p>This will not be bold</p>
```

Combining selectors

- You can combine selectors into lists

```
<style>
      .warning, h1 {color: red;}
</style>
```

Pseudo-classes and pseudo-elements

- You can also style certain states of an element or certain parts of elements

```
<style>
        a:hover {color: red;}
        p::first-line {color: blue;}
</style>
```

Descendent Selectors

- Elements that are descended from a particular element are styled according to the rule of the **descendent selector**
  - This means that the rules will be applied to a set of elements in one context but not in another

```
<style>
      p em { color: red; font-weight: bold;  }
</style>
<body>
      <p>this will be the default color
      <em>this will be coloured red and bold</em>
       back to the default colour</p>
</body>
```

Cascading Style Sheets

- You might specify **conflicting rules** in your CSS e.g. different values of the same property apply to the same element
- In this case it's important to understand how these rules get sorted out through **cascades**, **specificity** and **inheritance**

Inheritance of Style

- The order of application of styles is through inheritance
- Styles are applied not only to a specified element, but also to its descendants
  - For example, below will inherit the style of its parent <p>

```
<style>
      p { color: red;
          font-weight: bold  }
</style>
<body>
<p>happily grey <em>really emphasizing redness</em></p>
</body>
```

Specificity of Style

- Sometimes more than one rules apply to the same element
- When rules conflict, CSS uses **weights to decide which one applies**
  - **Adds a specificity weight**
- **Specificity order (highest → lowest):**
  - **ID selectors**
    `#header`
  - **Class / attribute / pseudo-class**
    `.menu`, `[type="text"]`, `:hover`
  - **Element selectors**
    `div`, `p`, `h1`, `::before`

```
h1 {color: red;}                    /* specificity = 0,0,1 */
body h1 {color: green;}             /* specificity = 0,0,2 */
#content h2 {color: silver;}        /* specificity = 1,0,1 */
h2.grape {color: purple;}           /* specificity = 0,1,1 */
h2 {color: silver;}                 /* specificity = 0,0,1 */
```

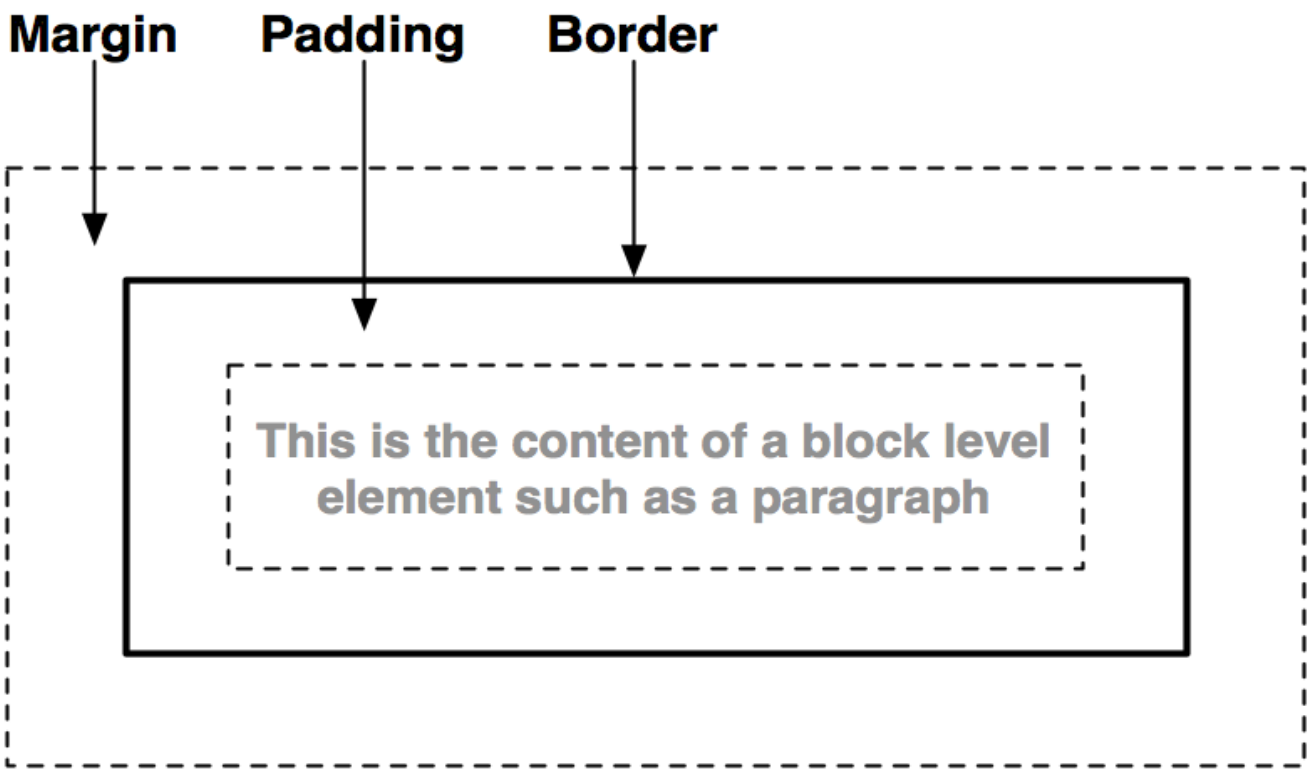
Calculator: https://specificity.keegan.st/

Cascade of Style

- However, sometimes there is still a conflict between two or more rules.
  - i.e. if they all have the same weight
- CSS is based on a method of causing **styles to cascade together**, which is made possible by combining inheritance, specificity and order
- The purpose of "**cascading**" is to find one winning rule among a set of rules that apply to a given element
- In essence, **if two rules have the same specificity, then the one that is defined last in the stylesheet wins**!

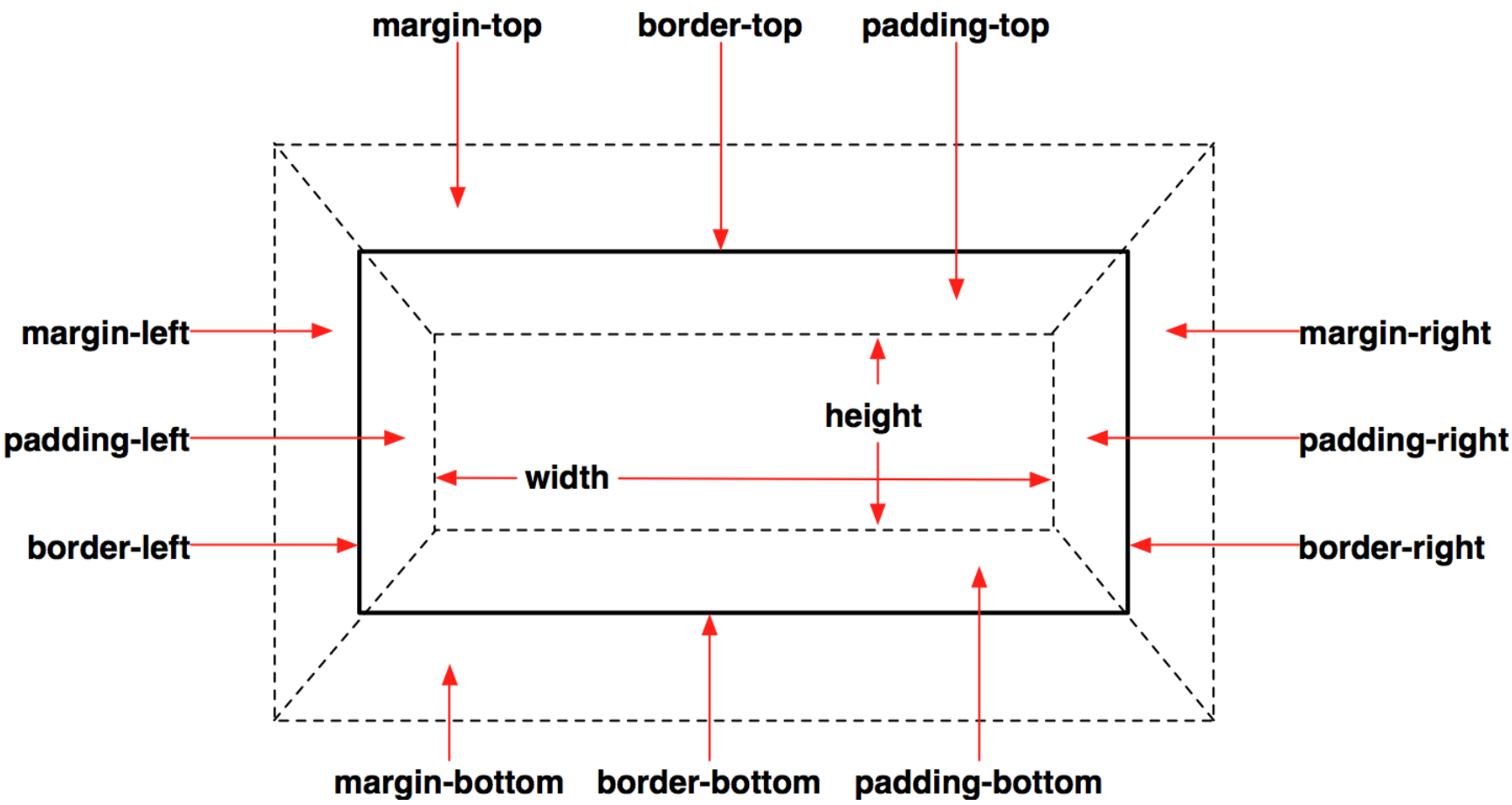
The Box Model

The Box Model



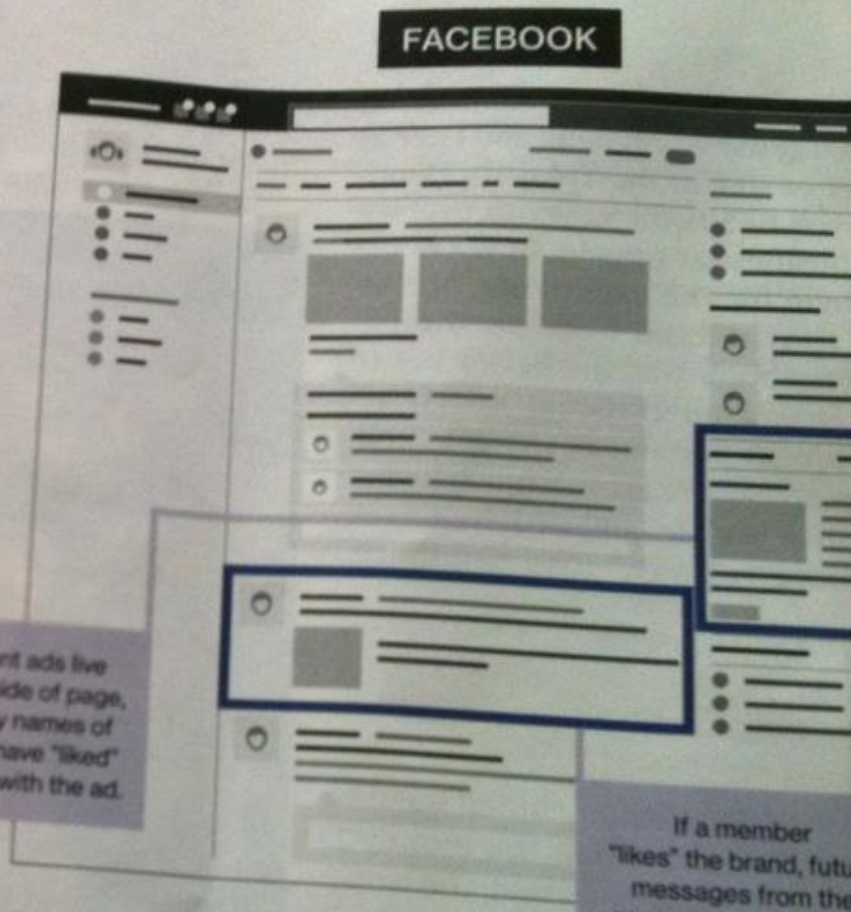
Every element generates one or more rectangular **element boxes** which houses the content. The element box is surrounded by optional amounts of **padding**, **borders**, and **margins**.

Box Model Properties



Page Layout

Sponsored links live on the right margin of the page and above search results marked by a colored background.



Engagement ads live on the right side of page, footnoted by names of friends who have "liked" or interacted with the ad.

If a member "likes" the brand, futu messages from the company might appe inside the member's newsfeed at no extr cost to the advertise

Promoted tweets show up in Twitter search results and will appear in regular Twitter feeds based partly on whom a member follows. Users can retweet ads at no extra cost to the advertiser.

as their product's "service missi
cial K's page dispenses nutritio
Nature Valley ruminates about
parks and nature photography.

Page Layout

- Layout of major elements on a webpage (e.g. columns, navigation bars, sidebars, headers and footers) can be specified using CSS
- In bygone days, tables were heavily used for layout
  - simple to use for simple tasks
  - painful for complex layout
  - tables are meant for content, not layout
  - not accessible for impaired users
- The preferred solution is to divide a page into a collection of (division/section) elements
  - ** … **
  - …

Floating

CSS floating properties allow you to **float** elements horizontally.

Elements can be floated: **left** and **right**, but not up and down!

Elements after the floating element will flow around. So if screen size changes elements will move down.

```
<head><style type="text/css" media="screen">
        .thumbnail {
                float:left;
                width:110px;
                height:90px;
                margin:5px;       }
  </style> </head>

<body>

<img class="thumbnail" src="klematis_small.jpg”>
<img class="thumbnail" src="klematis2_small.jpg”>
<img class="thumbnail" src="klematis3_small.jpg">

</body>
```



Positioning

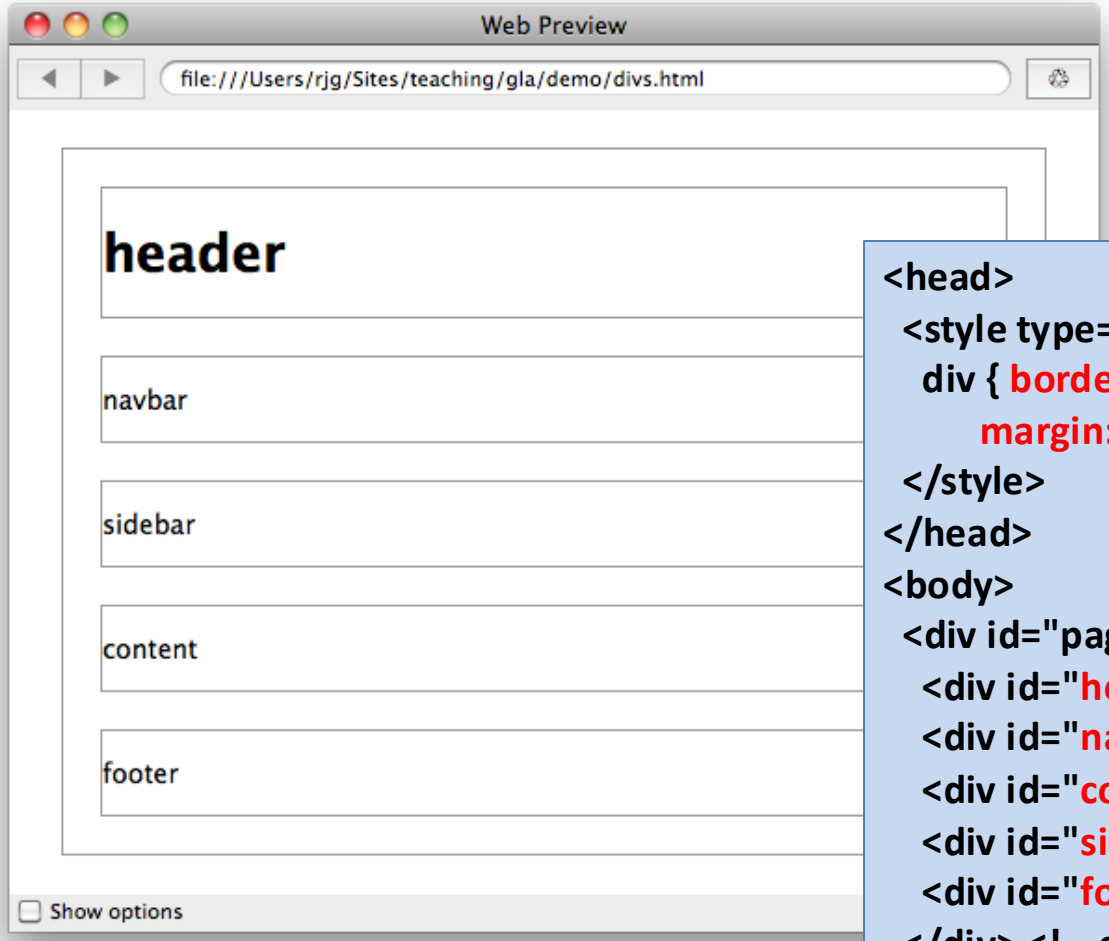
CSS positioning properties allow you to **position** an element.

Elements can be positioned using: **top**, **bottom**, **left** and **right** properties.

There are four different ways to position: static (default), **fixed**, **relative** and **absolute**.

```
<style type="text/css" media="screen">
        div {
                border: 1px solid #999;
                margin: 20px;    }

        div.fixed {
                position: fixed;
                top: 30px;
                right: 5px;        }

        div.Relative {
                position: relative;
                top: -50px;}

        div.Absolute {
                position: absolute;
                left: 100px;
                top: 150px;        }

 </style>
```

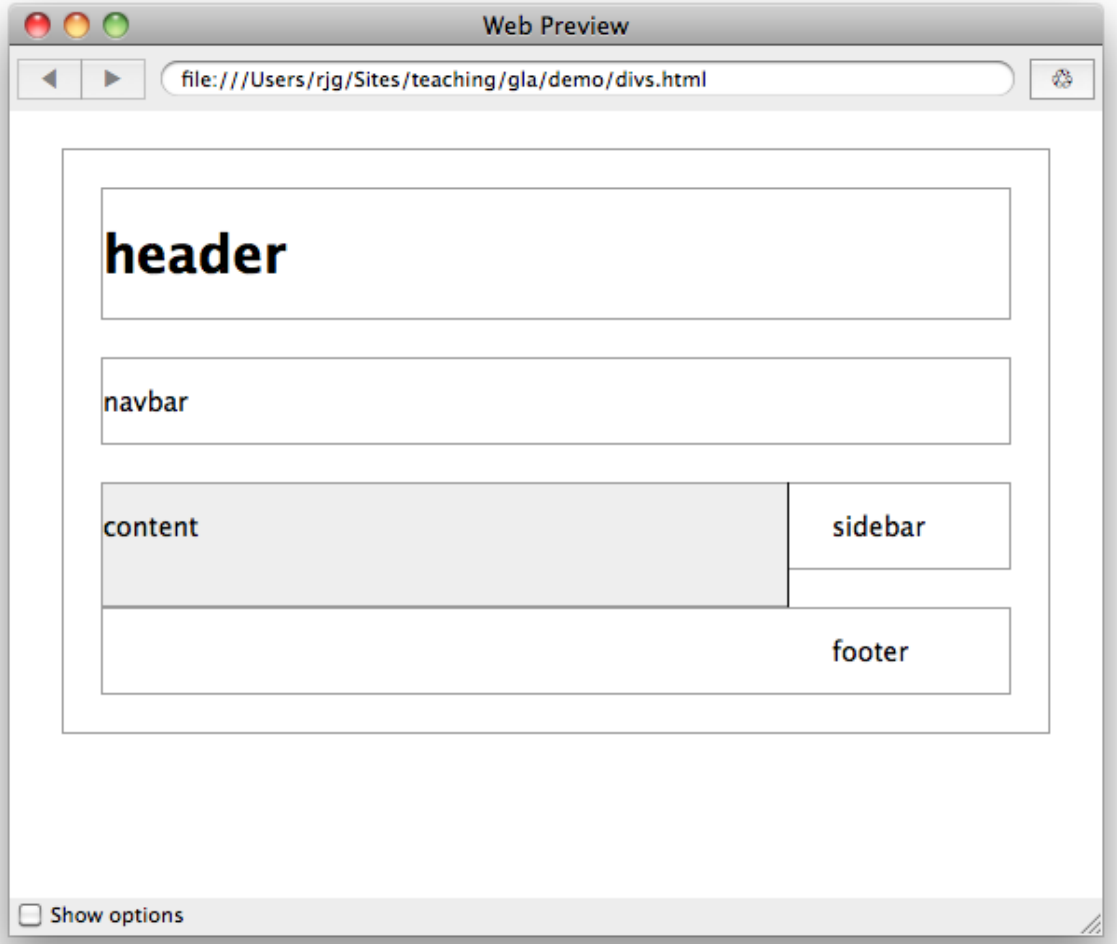
Floating & Positioning




```
<head>
 <style type="text/css" media="screen">
  div { border: 1px solid #999;
        margin: 20px;}
 </style>
</head>
<body>
 <div id="page">
  <div id="header"> <h1>header</h1> </div>
  <div id="navbar"> <p>navbar</p> </div>
  <div id="content"> <p>content</p> </div>
  <div id="sidebar"> <p>sidebar</p> </div>
  <div id="footer"> <p>footer</p> </div>
 </div> <!-- end of page -->
</body>
```

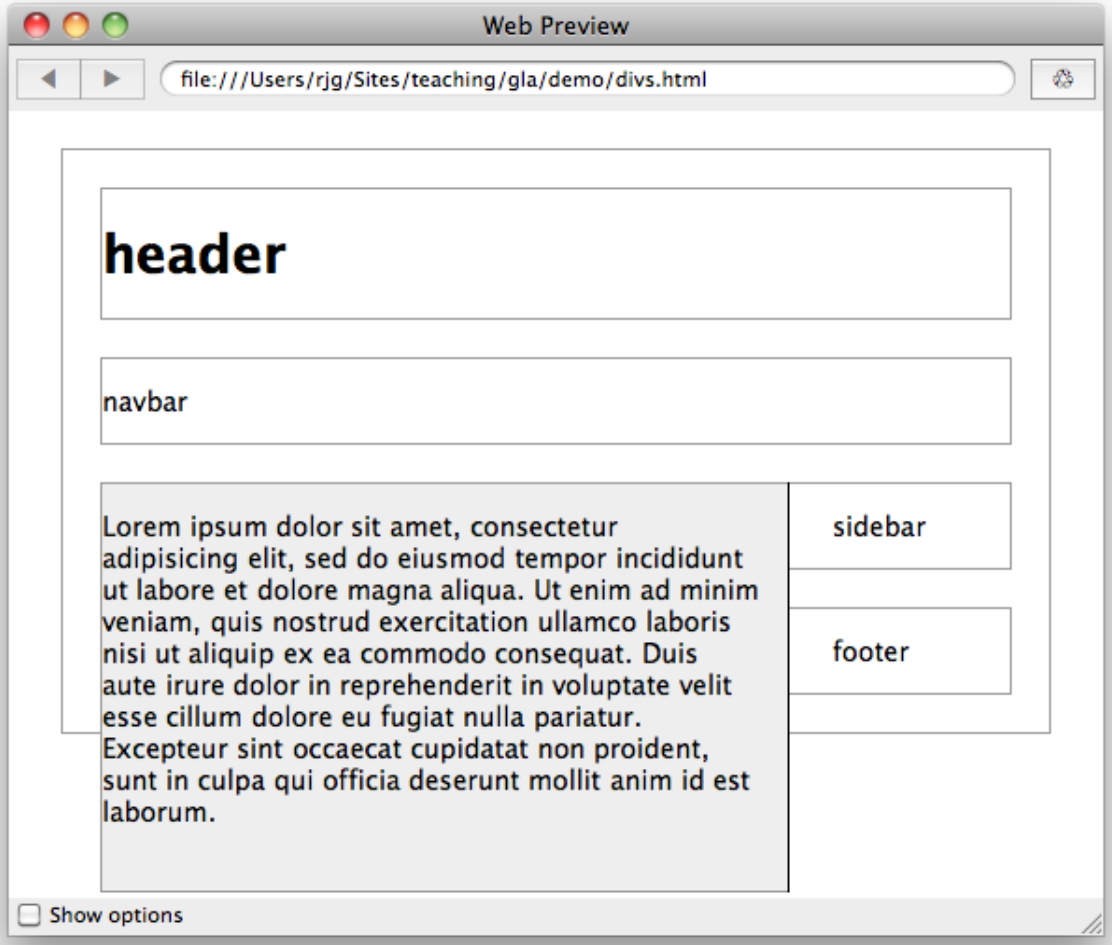
Floating and Positioning

```
#content {
    float: left;
    width: 67%;
    background: #eee;
    margin-top: 0;
    margin-right: 1.67em;
    border-right: 1px solid black;
    padding-top: 0;
    padding-right: 1em;
    padding-bottom: 20px;
  }
```



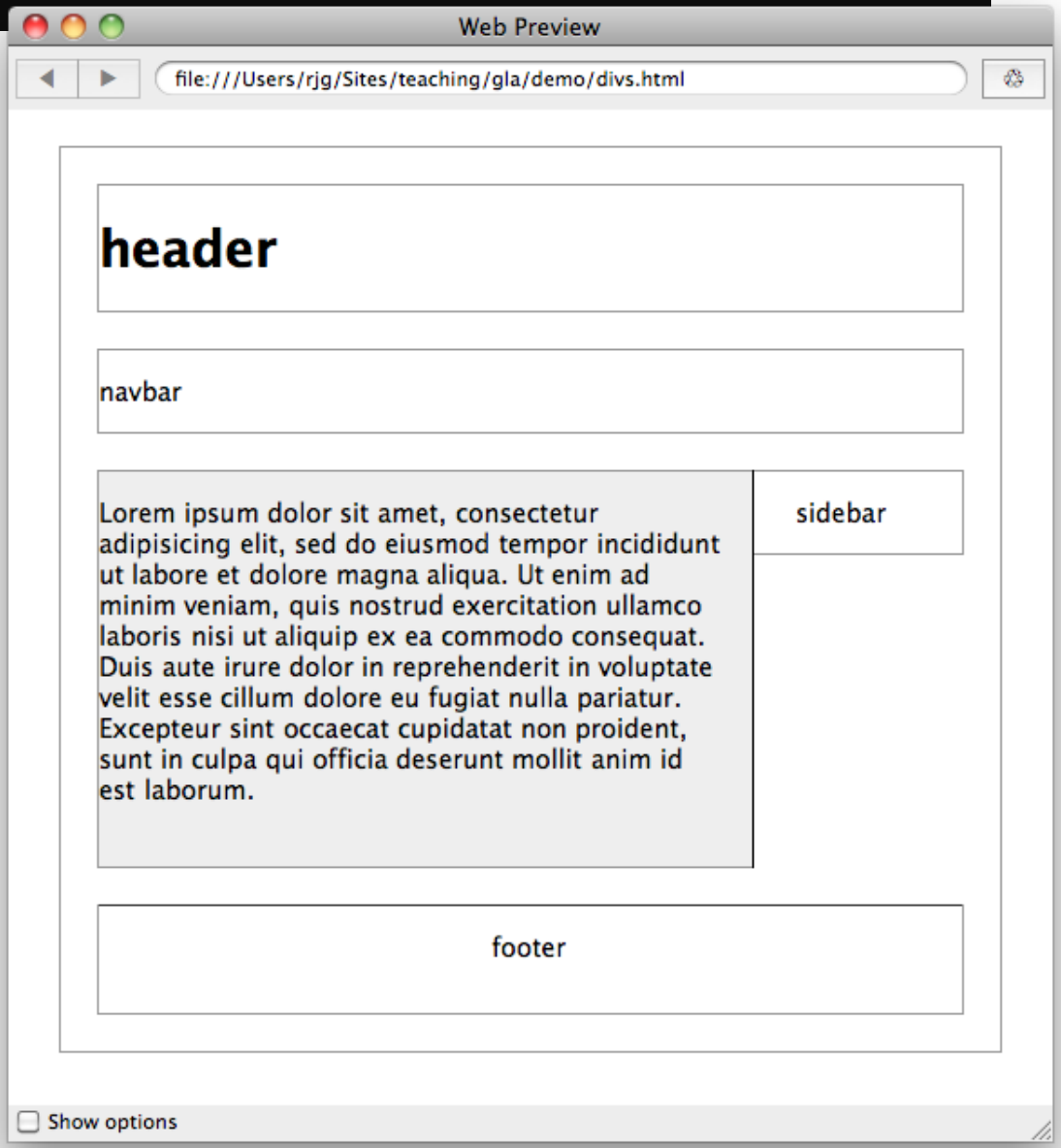
Floating and Positioning

Due to the natural flow layout, the footer block has appeared in the incorrect location.



Floating and Positioning

```
#footer {
    clear: both;
    padding-bottom: 1em;
    border-top: 1px solid #333;
    text-align: center;
   }
```



New Layouts

- Laying out pages with just floats and positioning is incredibly tricky and tedious
- Newer methods like Flexbox and Grids solve some of these issues
- These newer methods are also **responsive** so can deal with different screen sizes

CSS Properties

- The full list of properties can be found: https://www.w3.org/TR/CSS/#css
- Compact CSS Cheatsheets are useful: http://www.lesliefranke.com/files/reference/csscheatsheet.html

background-color	**border-width**	**font-family**	**height**	**size**
text-align	**width**	**color**	**font-size**	**margin**
padding	**list-style**	**position**	**text-decorations**	**~100+ more!**

Web sustainability and CSS

- CSS helps avoid duplication
  - One CSS class to handle everything
- CSS helps ease of management and code readability
- CSS makes it easier to design responsive content for different devices

Web sustainability and CSS

Guideline #3.6: Avoid code duplication

- CSS add more code to your markup, but improve maintainability

- Repetitive code increases energy

Web sustainability and CSS

Guideline #3.6: Avoid code duplication

- CSS add more code to your markup, but improve maintainability

- Repetitive code increases energy

How should I structure my CSS?

- Write CSS for sections or files
- Search for duplicate declarations
  - Check order
  - Determine which rule should come first
  - Remove duplicate declarations

Web sustainability and CSS

Guideline #3.13: Adapt to User Preferences

- Users often have preferences that are more environmentally-friendly

- CSS media/preference queries should enable users, e.g. `monochrome`, `prefers-reduced-data`, `prefers-reduced motion`, `prefers-color-scheme`

Web sustainability and CSS

Guideline #3.14: Develop a Device-Adaptable Layout

- Many different types of devices: desktops/laptops, mobiles, tablets, watches, TVs with different screen sizes, memory, power/energy require appropriate layouts

- Smaller devices are underpowered

Device-adaptable layouts

- Responsive design:
  - Fluid pages that adapt to screen size
  - Modern CSS layout methods are responsive (Flexbox, Grid)
- Optimized photos, graphics, media
  - Resize images for different devices, reduce page weight
    - Images are ~50% of a desktop page weight
- Dark mode - proceed with caution!
  https://www.businessinsider.com/guides/tech/does-dark-mode-save-battery?r=US&IR=T

Mobile First

Mobile First: A guideline for web designers to aid sustainability

- Determine a budget for your page weight

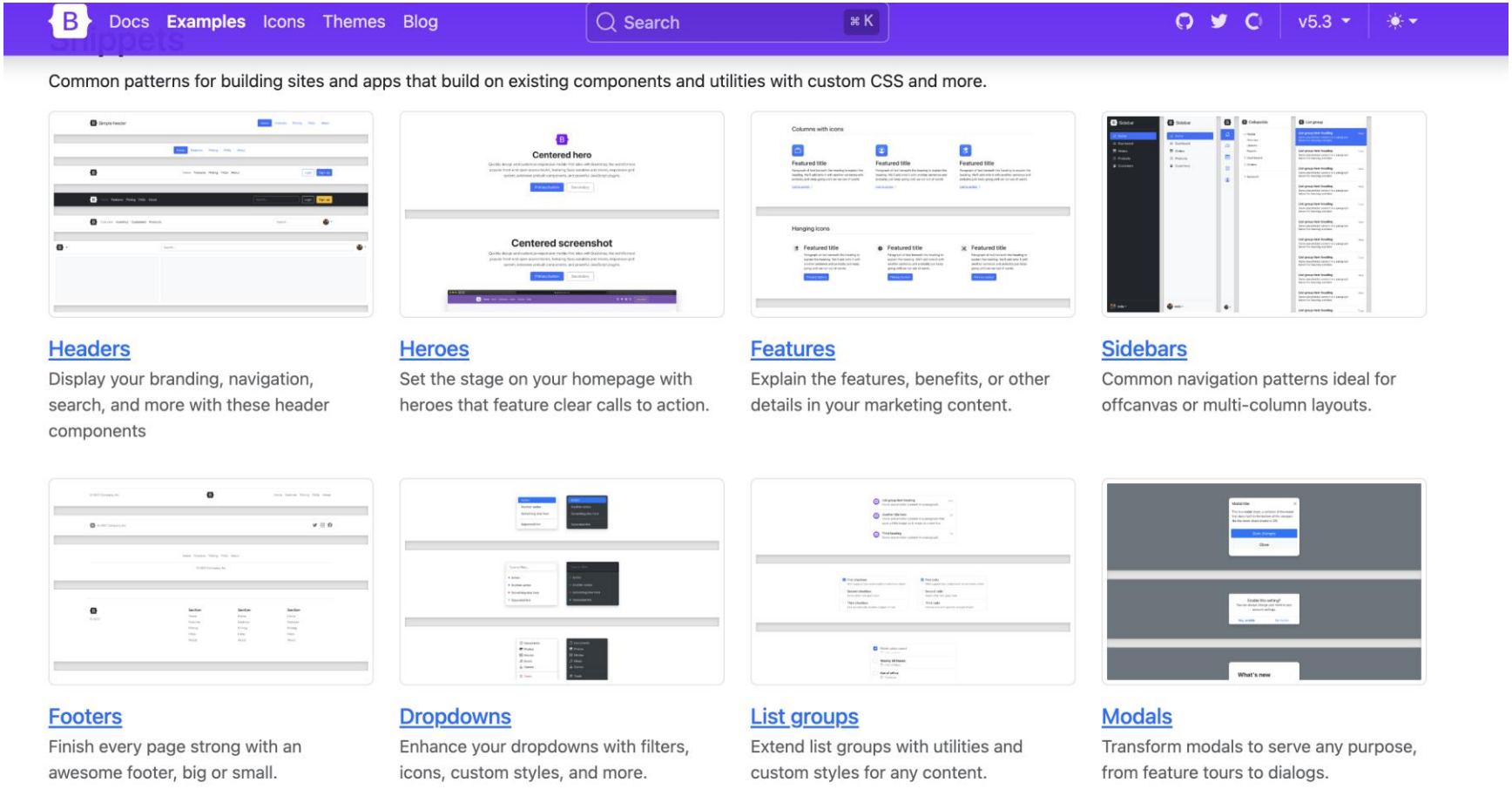
The median webpage is 2.3MB
– 600% up in the last 10 years

- Optimize media
- Remove unnecessary elements
- Minify HTML, CSS, Javascript

What is Bootstrap?

- Another web application framework (WAF) **for front-end development**
- Very commonly used toolkit for designing and styling responsive web applications
- Provides layouts, containers, elements, examples, themes (and JavaScript!)
- Advantages
  - Easy to use
  - Responsive features
  - Mobile-first approach
  - Cross-browser compatibility

Boostrap examples



Using Bootstrap

- https://getbootstrap.com/ provides different releases. Latest is 5.*, TWD book uses 4

- Go to w3schools and follow the tutorial (also other good resources out there)

- Download an example and adapt it

Summary

- Covered two major building blocks of web pages
- HTML is the basic building block of the web using a markup language to provide content and structure
- CSS is a powerful method of specifying the style of web pages
  - separates presentation from structure and content
- Bootstrap is a toolkit to help design responsive web pages